

MÓDULO PROFESIONAL: ACCESO A DATOS (AD - 0486)

UT1: Manejo de Ficheros en Java

Persistencia Física, Flujos de Caracteres, Java NIO.2, Formatos Binarios Tipados, RandomAccessFile, Serialización de Objetos y Lenguajes Estructurados XML (DOM, SAX y JAXB)

Bloque 1 · Texto y NIO.2

Streams, Búferes, Path, Files, Charsets y lecturas masivas perezosas.

Bloque 2 · Binarios y RAF

DataStream, acceso aleatorio por offset en disco y serialización con transient.

Bloque 3 · Documentos XML

Árbol en memoria con DOM, lectura reactiva SAX y Binding automático JAXB.



Mapa Conceptual: ¿Por qué gestionamos ficheros?

No toda la información reside en bases de datos. Los ficheros son indispensables para:



Persistencia Local Rápida

Configuraciones de la aplicación (`.properties` , `.env`),
cachés locales en disco y estados de sesión.



Ingesta Masiva y Logs

Trazabilidad de servidores web (Apache/Nginx), pipelines de
datos CSV e intercambio estructurado B2B con XML.



Objetivo Profesional: Aprender a seleccionar la técnica exacta según el volumen, tipo de dato y requisitos de latencia y memoria RAM.

1. El Concepto de Stream (Flujo) en Java

Un **Stream** es un conducto abstracto secuencial entre una fuente y un destino, aislando el hardware del código.

FAMILIA DE FLUJOS	CLASES RAÍZ	UNIDAD ATÓMICA	CASOS DE USO PRIMORDIALES
Flujos de Bytes	<code>InputStream</code> / <code>OutputStream</code>	8 bits (Byte binario)	Ficheros binarios, imágenes, audio, PDFs, red
Flujos de Caracteres	<code>Reader</code> / <code>Writer</code>	16 bits (Carácter Unicode)	Texto plano, archivos de configuración, CSV, JSON

 **Regla de Oro:** Los `Reader/Writer` traducen bytes a caracteres usando una tabla de codificación (*Charset*). Nunca leas un binario con un Reader: corromperás los datos.

⚡ 2. Búferes y el Patrón Decorador

Acceder al disco físico carácter a carácter colapsa la CPU con interrupciones del SO.

```
// Patrón Decorador: BufferedReader decora a FileReader aportando un buffer de 8 KB
try (BufferedReader br = new BufferedReader(new FileReader("datos.txt"))) {
    String linea;
    // readLine() lee bloques de disco a RAM y retorna null al llegar a EOF
    while ((linea = br.readLine()) != null) {
        System.out.println(linea);
    }
}
```

💡 `readLine()` consume el salto de línea pero **no lo incluye**. Para escribir texto con saltos de línea consistentes entre Windows y Linux, usa siempre `bw.newLine()` .

3. La Revolución de Java NIO.2 (`java.nio.file`)

Reemplaza las limitaciones históricas de `java.io.File` con operaciones atómicas y rutas limpias:

Path & Paths

Representación inmutable y multiplataforma de rutas sin acoplar barras invertidas de Windows.

```
Path p = Paths.get("data", "logs", "app.log");  
System.out.println(p.toAbsolutePath());
```

Files

Operaciones atómicas completas en una sola llamada:

```
Files.exists(p);  
Files.createDirectories(p.getParent());  
Files.copy(origen, destino, StandardCopyOption.REPLACE_EXISTING);
```

🔥 4. En Producción: `readAllLines()` vs `Files.lines()`

MÉTODO	RETORNO	CONSUMO RAM	RIESGO	USO RECOMENDADO
<code>Files.readAllLines()</code>	<code>List<String></code>	$O(N)$ (Carga todo en RAM)	<code>OutOfMemoryError</code> en archivos grandes	Configs y CSVs pequeños (< 20 MB)
<code>Files.lines()</code>	<code>Stream<String></code>	$O(1)$ (Perezoso / Lazy)	Ninguno (cierre obligatorio)	Logs y ficheros masivos de Gigabytes

```
// Procesa 10 GB consumiendo solo 25 MB de memoria RAM
try (Stream<String> stream = Files.lines(logPath, StandardCharsets.UTF_8)) {
    long errores = stream.filter(l -> l.contains(" 500 ")).count();
}
```

5. Ficheros Binarios Tipados: `DataInputStream/OutputStream`

Guardar números como texto plano ("12345678") gasta 8 bytes y exige parseo. En binario, un entero ocupa siempre **4 bytes exactos**.

Escritura

```
try (DataOutputStream dos = new DataOutputStream(  
    new FileOutputStream("sensor.dat"))) {  
    dos.writeInt(101);           // 4 bytes  
    dos.writeDouble(23.75);      // 8 bytes  
    dos.writeUTF("ACTIVO");      // Longitud + UTF-8  
}
```

Lectura Secuencial

```
try (DataInputStream dis = new DataInputStream(  
    new FileInputStream("sensor.dat"))) {  
    int id = dis.readInt();  
    double temp = dis.readDouble();  
    String st = dis.readUTF();  
}
```

 **Regla Inquebrantable:** Se deben leer los tipos **exactamente en el mismo orden** en que fueron grabados. El fin de archivo se detecta capturando `EOFException`.

6. Ficheros de Acceso Aleatorio: `RandomAccessFile`

Trata el fichero como un gran array de bytes en disco. Permite saltar a cualquier posición sin leer el archivo desde el inicio.

- Modos de apertura: `"r"` (solo lectura) y `"rw"` (lectura y escritura).
- Puntero de Fichero: `seek(posicionBytes)` y `getFilePointer()` .

```
try (RandomAccessFile raf = new RandomAccessFile("empleados.bin", "rw")) {  
    // Si cada registro mide exactamente 50 bytes:  
    int idEmpleado = 3;  
    long offset = (idEmpleado - 1) * 50L;  
  
    raf.seek(offset);                // Salto directo en nanosegundos  
    int salarioActual = raf.readInt();  
  
    raf.seek(offset);                // Reubica el puntero para modificar in-situ  
    raf.writeInt(salarioActual + 200); // Actualización sin reescribir el resto del fichero  
}
```

7. Serialización de Objetos en Java

Convierte el grafo completo de un objeto en memoria en una secuencia lineal de bytes transportable o almacenable.

Requisitos de la Entidad

- Implementar `java.io.Serializable`.
- Definir `serialVersionUID = 1L;`.
- Usar `transient` para campos que NO deban guardarse (passwords, conexiones).

Canales ObjectOutputStream

```
// Escritura
ObjectOutputStream oos = new ObjectOutputStream(fos);
oos.writeObject(listaAlumnos);

// Lectura (Deserialización)
ObjectInputStream ois = new ObjectInputStream(fis);
List<Alumno> list = (List<Alumno>) ois.readObject();
```

⚠ Si cambias la clase sin definir `serialVersionUID` explícito, Java lanzará `InvalidClassException` al leer datos guardados anteriormente.

8. Tratamiento de XML en Java: Los 3 Enfoques

DOM

Document Object Model

Carga el documento completo en memoria en un árbol de nodos navegable. Ideal para consultar y modificar ficheros medianos (< 20 MB).

SAX

Simple API for XML

Basado en eventos y callbacks (*startElement*, *endElement*). Memoria constante $O(1)$. Solo lectura secuencial para XMLs masivos.

JAXB

Java Architecture for XML Binding

Mapeo declarativo mediante anotaciones (`@XmlRootElement`). Convierte POJOs a XML (*Marshaller*) automáticamente.

9. Lectura y Escritura con DOM (`DocumentBuilder` & `Transformer`)

```
// 1. Construir el árbol en memoria RAM
DocumentBuilder builder = DocumentBuilderFactory.newInstance().newDocumentBuilder();
Document doc = builder.parse(new File("alumnos.xml"));
doc.getDocumentElement().normalize();

// 2. Navegación por etiquetas
NodeList lista = doc.getElementsByTagName("alumno");
for (int i = 0; i < lista.getLength(); i++) {
    Element elem = (Element) lista.item(i);
    System.out.println(elem.getAttribute("dni") + " - " +
        elem.getElementsByTagName("nombre").item(0).getTextContent());
}

// 3. Volcado a disco con Transformer
Transformer t = TransformerFactory.newInstance().newTransformer();
t.setOutputProperty(OutputKeys.INDENT, "yes");
t.transform(new DOMSource(doc), new StreamResult(new File("salida.xml")));
```

⚡ 10. SAX por Eventos y JAXB Declarativo

SAX: `DefaultHandler`

No carga nada en RAM; avisa al encontrar etiquetas:

```
class MiHandler extends DefaultHandler {  
    public void startElement(String uri, String localName,  
                             String qName, Attributes atts) {  
        if (qName.equals("alumno")) {  
            System.out.println(atts.getValue("dni"));  
        }  
    }  
}
```

JAXB: Mapeo Automático

Sin manipular nodos XML manualmente:

```
@XmlRootElement  
public class Alumno {  
    @XmlAttribute private String dni;  
    @XmlElement private String nombre;  
}  
  
// Marshaller: Objeto -> XML  
JAXBContext ctx = JAXBContext.newInstance(Alumno.class);  
ctx.createMarshaller().marshal(alumno, new File("alumno.xml"));
```

 ¿Cuándo usar cada tecnología en proyectos reales?

TECNOLOGÍA	TIPO DE DATOS	VENTAJA CLAVE	ELÍGELA CUANDO...
NIO.2 (Files/Path)	Texto plano / CSV	Expresiva y atómica	Configuración, scripts y logs masivos (<code>lines()</code>)
DataStream	Binario primitivo	Compacto y rápido	Sensores, telecomunicaciones, registros de tamaño fijo
RandomAccessFile	Binario aleatorio	Edición directa por offset	Motores de bases de datos embebidos e índices
Serialización	Objetos Java	Guarda grafos completos	Caché en disco y estado de sesiones en RAM
DOM	Documentos XML	Árbol modificable	Ficheros XML pequeños o medianos
SAX / JAXB	XML masivo / POJOs	Memoria $O(1)$ / Agilidad	Grandes catálogos XML / Integración con APIs



Práctica de Afianzamiento de UT1

Proyecto: CLI de Migración y Gestión Multi-Formato

Durante las próximas sesiones aplicaremos todos los conceptos integrando:

- Limpieza y filtrado seguro de ficheros CSV con `Files.lines()` .
- Acceso y actualización directa de stock mediante `RandomAccessFile` .
- Copia de seguridad serializada con atributos `transient` protegidos.
- Exportación y consulta estructurada del catálogo en `XML` mediante DOM.

¡Dominando la persistencia desde los cimientos! 